

GRAND PROJET INTÉGRATEUR DOCTORAL TP 2 : Du Calcul Symbolique à l'IA Scientifique Multi-GPU avec Automatisation de la Reproductibilité (CI/CD)

Cadre Institutionnel et Modalités d'Évaluation

Ce projet constitue le jalon central d'évaluation de vos compétences en ingénierie de recherche et calcul haute performance.

- **Reproductibilité Absolue (No-Sudo)** : Aucun script, test ou workflow ne doit invoquer `sudo`. Vos pipelines doivent s'exécuter dans des espaces utilisateurs restreints, préfigurant les contraintes des supercalculateurs. Aucun droit `root` requis. Environnement scellé de bout en bout.
- **Élite Qualimétrique** : Validation stricte exigée par les linters sans tolérance d'erreur, et couverture minimale par les tests de 85% (90% pour le projet final).
- **Livrable attendu** : Un lien unique vers un dépôt GitHub (privé avec accès accordé aux évaluateurs, ou public) contenant le code source intégralement typé, la suite de tests unitaires, le workflow d'automatisation exécutable, et le rapport final.
- **Notation** : Évaluée sur la base du succès des pastilles de statut (*Build Badges*) et de l'historique des déclenchements de la CI, ainsi que sur la rigueur scientifique, la robustesse logicielle et la performance computationnelle.

1 Introduction et Problématique Scientifique

Dans l'écosystème de la recherche moderne, la validation des résultats ne peut plus reposer sur l'affirmation « *Ça fonctionne sur ma machine* ». L'automatisation des tests sur une infrastructure tierce (Intégration Continue ou CI) est le seul moyen de garantir la non-régression d'un code scientifique lors de l'ajout de nouveaux modules mathématiques ou de modifications de l'infrastructure logicielle.

Ce projet simule l'étude d'un phénomène de transfert de masse régi par une équation différentielle non linéaire de type Advection-Diffusion-Réaction singulière. Pour résoudre et analyser ce système, vous mettrez en place un pipeline de recherche moderne combinant :

- Calcul symbolique et manipulation de données massives
- Accélération de bas niveau et distribution de charge
- Deep learning informé par la physique (PINN)
- Automatisation complète via GitHub Actions et orchestration par workflow

L'objectif est de concevoir un workflow **GitHub Actions** capable de valider de manière autonome ce pipeline de calcul scientifique, en garantissant l'étanchéité des versions, la qualité de l'architecture et la génération automatisée d'artéfacts de résultats.

2 Module 0 : Architecture Logicielle et Prérequis CI/CD

Avant de configurer l'infrastructure de CI et de développer les modules scientifiques, le projet doit posséder les mécanismes d'auto-validation standards.

2.1 Exercice 0.1 : Structure minimale du dépôt

Initialisez un dépôt Git local et configurez l'arborescence générique suivante :

```
phd_integrator_project/
├── .github/
│   └── workflows/
│       └── ci_cd_pipeline.yml <- Fichier de configuration GitHub Actions
├── data/
│   └── raw_sensors/
├── src/
│   ├── __init__.py
│   ├── symbolic_derivation.py <- Calcul symbolique (Module 2)
│   ├── numerical_core.py <- Manipulation NumPy (Module 3)
│   ├── hpc_acceleration.py <- Optimisation Numba (Module 5)
│   └── deep_pinn.py <- Réseau de neurones PINN (Module 6)
├── tests/
│   └── test_core.py <- Suite de tests unitaires (PyTest)
├── outputs/
│   └── figures/
├── pyproject.toml <- Déclaration des dépendances (PEP 518)
├── uv.lock <- Verrouillage déterministe
├── Snakefile <- Orchestration Snakemake
└── README.md
```

2.2 Exercice 0.2 : Isolation et tests locaux avec uv

Configurez votre fichier `pyproject.toml` en intégrant les outils de validation statique et dynamique (`pytest`, `pytest-cov`, `mypy`, `ruff`).

Ajoutez les dépendances strictes suivantes : `numpy`, `scipy`, `sympy`, `polars`, `matplotlib`, `seaborn`, `plotly`, `numba`, `torch`, `snakemake`, `pytest`, `pytest-cov`, `ruff`, `mypy`.

Générez votre fichier de verrouillage déterministe `uv.lock` localement. Vérifiez que votre suite de tests passe avec succès en local via l'environnement virtuel isolé.

3 Module 1 : Configuration du Workflow GitHub Actions

Les GitHub Actions utilisent des exécuteurs distants (*Runners*) configurés via des scripts au format YAML.

3.1 Exercice 1.1 : Déclaration des déclencheurs et de l'environnement

Créez le fichier `.github/workflows/ci_cd_pipeline.yml`. Nous souhaitons que le pipeline s'exécute de manière automatique à chaque fois qu'un chercheur pousse du code (`push`) sur la branche `main`, ou lorsqu'une demande de fusion (`pull_request`) est soumise.

3.2 Exercice 1.2 : Rédaction des étapes de validation (Jobs)

Complétez le fichier de configuration YAML afin d'enchaîner les étapes indispensables de l'intégration continue. **Contrainte obligatoire** : Vous devez exploiter l'action officielle d'installation d'`uv`

pour garantir l'absence totale d'installation globale lente ou nécessitant des privilèges root.

Basez-vous sur le squelette rigoureux ci-dessous :

```
1 name: Scientific Computing CI/CD Pipeline
2
3 on:
4   push:
5     branches: [ main ]
6   pull_request:
7     branches: [ main ]
8
9 jobs:
10  validate-and-test:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Code Checkout
15        uses: actions/checkout@v4
16
17      - name: Install uv (Rust-backed package manager)
18        uses: astral-sh/setup-uv@v5
19        with:
20          version: "latest"
21          enable-cache: true
22
23      - name: Set up Python
24        run: uv python install 3.11
25
26      - name: Static Analysis - Linting (Ruff)
27        run: uv run ruff check src/ tests/
28
29      - name: Static Analysis - Type Checking (Mypy)
30        run: uv run mypy --strict src/
31
32      - name: Execute Test Suite & Coverage (PyTest)
33        run: uv run pytest --cov=src --cov-report=xml tests/
```

Listing 1 – Squelette du pipeline CI/CD GitHub Actions

3.3 Questions d'analyse théorique

1. Quelle est l'utilité de l'option `enable-cache: true` spécifiée dans l'étape d'installation d'`uv` ? Quel est son impact direct sur l'empreinte carbone et le coût computationnel d'une infrastructure de recherche ?
2. Expliquez pourquoi l'étape de vérification de types (`mypy -strict`) est indispensable lors de la manipulation de matrices de données complexes avant de lancer des simulations de plusieurs heures sur un supercalculateur.

4 Module 2 : Déploiement Continu (CD) et Gestion des Artéfacts

L'intégration continue ne se limite pas à valider le code ; elle doit également publier de manière sécurisée les artéfacts scientifiques produits (modèles de réseaux de neurones sauvegardés, figures de résultats vectorielles ou rapports d'exécution au format PDF).

4.1 Exercice 2.1 : Persistance des artéfacts de calcul

Modifiez votre workflow YAML pour ajouter une étape finale s'exécutant uniquement en cas de succès des étapes précédentes (`success()`). Cette étape devra utiliser l'action `actions/upload-artifact`

pour extraire le répertoire contenant vos figures de résultats (`outputs/figures/`) ou vos fichiers de poids de modèles afin de les rendre téléchargeables directement depuis l'interface GitHub.

4.2 Exercice 2.2 : Matrice de tests multi-environnements (Matrix Build)

En recherche computationnelle, un code doit souvent être validé sur plusieurs architectures ou plusieurs versions d'interpréteurs. Modifiez la section `jobs` de votre fichier YAML pour introduire le mot-clé `strategy: matrix`. Configurez le pipeline pour qu'il lance en parallèle trois exécuteurs distincts testant votre code sur Python 3.10, 3.11 et 3.12.

Espace réponse : Configuration de la stratégie matricielle YAML

Rédigez ici le bloc YAML exact définissant la stratégie de matrice pour les versions de Python.

5 Module 3 : Calcul Symbolique et Fondamentaux Python

Dans le fichier `src/symbolic_derivation.py`, concevez des fonctions Python pures et modulaires.

1. À l'aide de `SymPy`, définissez les variables symboliques x , t , ainsi que les paramètres c (advection) et ν (diffusion).
2. Déclarez une solution analytique candidate de type onde solitaire : $u(x, t) = \tanh(x - c \cdot t)$.
3. Calculez symboliquement les dérivées partielles $\frac{\partial u}{\partial t}$, $\frac{\partial u}{\partial x}$ et $\frac{\partial^2 u}{\partial x^2}$.
4. Injectez ces expressions dans l'équation d'advection-diffusion afin de générer automatiquement le terme source résiduel exact $f(x, t) = \frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} - \nu \frac{\partial^2 u}{\partial x^2}$.
5. Exportez ces fonctions sous forme de code NumPy optimisé via la fonction `sympy.lambdify`.

6 Module 4 : Maîtrise de NumPy et Ingestion de Données

6.1 Exercice 4.1 : Manipulation Bas Niveau du ndarray

Dans `src/numerical_core.py`, manipulez la mémoire des structures de données :

1. Créez une grille bidimensionnelle spatio-temporelle (X, T) de taille $N \times M$.
2. Explorez et affichez explicitement les attributs `shape`, `dtype` et `strides` de vos tableaux.
3. **Layout Mémoire** : Modifiez l'agencement mémoire pour comparer un stockage contigu en ligne (C-contiguous) et en colonne (F-contiguous). Expliquez l'impact de ces layouts sur la localité du cache CPU lors des réductions par ligne ou par colonne.
4. **Vues vs Copies** : Réalisez une opération de découpage (*slicing*) sur la grille. Démontrez par programmation (en vérifiant la base mémoire avec `.base`) s'il s'agit d'une simple vue ou d'une copie effective en mémoire.

6.2 Exercice 4.2 : Vectorisation, Broadcasting et Ingestion Massives

Le dossier `data/raw_sensors/` contient des fichiers massifs de mesures climatiques au format Parquet ou de grands fichiers CSV.

1. Utilisez l'API Lazy de `Polars` (ou `DuckDB`) pour scanner et filtrer ces fichiers de données sans encombrer la RAM. Extrayez les coordonnées géospatiales valides sous forme de vecteurs NumPy.
 2. Bannissez toute boucle `for` itérative de vos calculs numériques. Utilisez les mécanismes de **broadcasting** de NumPy pour appliquer la fonction résiduelle $f(x, t)$ calculée au Module 3 sur l'ensemble de la grille de coordonnées de manière instantanée et vectorisée.
-

7 Module 5 : Stabilité, Conditionnement et Analyse des Erreurs

Les approximations de la norme IEEE 754 introduisent des dérives critiques. Nous étudions ici la sensibilité de la résolution d'un système linéaire issu de la discrétisation spatiale : $A \cdot \alpha = b$, où A est une matrice d'interpolation de type Hilbert ou RBF (Radial Basis Function).

7.1 Exercice 5.1 : Sensibilité et Précision des Types Numériques

1. Construisez la matrice A de taille $n \times n$. Évaluez son nombre de conditionnement $\kappa(A) = \|A\| \cdot \|A^{-1}\|$ en utilisant `numpy.linalg.cond`.
 2. Faites varier la dimension n (de 5 à 25). Mesurez l'erreur de reconstruction du système en comparant une implémentation en précision réduite (`float16`), en simple précision (`float32`) et en double précision (`float64`).
 3. **Propagation des erreurs et Perturbations** : Introduisez une perturbation aléatoire d'ordre $\epsilon = 10^{-7}$ sur le vecteur de second membre b . Mesurez l'impact de cette perturbation sur la solution α . Reliez mathématiquement cette amplification d'erreur à la valeur de $\kappa(A)$.
 4. **Validation Numérique** : Rédigez une fonction de validation calculant le résidu strict $r = \|A\alpha - b\|$. Pour vos assertions de tests, justifiez pourquoi l'usage d'égalités strictes (`==`) est proscrit et implémentez une validation robuste à l'aide de `np.isclose()` en configurant des tolérances (`atol`, `rtol`) adaptées à la précision choisie.
-

8 Module 6 : Optimisation, Accélération et Parallélisme

Dans le fichier `src/hpc_acceleration.py`, nous cherchons à optimiser un opérateur de filtrage local lourd appliqué sur notre grille numérique, défini par des boucles imbriquées complexes.

8.1 Exercice 6.1 : Diagnostic de Profiling

Utilisez les outils `cProfile` ou `py-spy` (profiling statistique) combinés au module `timeit` pour localiser et quantifier précisément le goulot d'étranglement de votre code initial non optimisé. Insérez l'analyse des temps d'appels dans votre rapport.

8.2 Exercice 6.2 : Accélération JIT via Numba

Appliquez le décorateur `@njit` de Numba sur votre fonction de filtrage.

1. Activez les options `parallel=True` (pour forcer la vectorisation et l'utilisation de boucles parallèles multi-cœurs via `prange`) et `fastmath=True`.

2. Analysez théoriquement et empiriquement le gain de performance obtenu. Expliquez les risques mathématiques induits par le drapeau `fastmath=True` vis-à-vis des strictes contraintes de la norme IEEE 754.

8.3 Exercice 6.3 : Parallélisation de haut niveau

Pour explorer un large espace de paramètres (par exemple, tester 100 combinaisons de coefficients c et ν), mettez en oeuvre une parallélisation multi-processus à l'aide de `Joblib`, du module natif `multiprocessing`, ou d'un cluster local `Dask` / `Ray`. Comparez les temps d'exécution globaux en fonction du nombre de workers alloués.

9 Module 7 : Calcul Haute Performance et IA Scientifique

Nous abordons la modélisation avancée en remplaçant les solveurs classiques par un réseau de neurones informé par la physique (**Physics-Informed Neural Network - PINN**) visant à approximer la solution de notre équation différentielle.

9.1 Exercice 7.1 : Architecture et Différentiation Automatique sous PyTorch / JAX

Dans `src/deep_pinn.py`, implémentez la structure du réseau de neurones en utilisant `PyTorch` ou `JAX`.

1. Concevez un perceptron multicouche (*MLP*) prenant en entrée les variables physiques (x, t) et retournant l'approximation $\hat{u}(x, t)$.
2. **Perte Physique sans maillage** : Exploitez le moteur de différentiation automatique (`torch.autograd.grad` ou `jax.grad`) pour calculer explicitement les opérateurs différentiels d'ordre supérieur du résiduel physique de l'équation :

$$\mathcal{L}_{\text{physique}} = \frac{1}{N_p} \sum_{i=1}^{N_p} \left(\frac{\partial \hat{u}}{\partial t} + c \frac{\partial \hat{u}}{\partial x} - \nu \frac{\partial^2 \hat{u}}{\partial x^2} - f(x_i, t_i) \right)^2 \quad (1)$$

3. Combinez cette perte avec la perte sur les conditions aux limites et les données issues des capteurs ($\mathcal{L}_{\text{données}}$) pour former la fonction de perte totale.

9.2 Exercice 7.2 : Abstraction et Exécution Multi-GPU

Configurez le code pour détecter et exploiter de manière optimale les architectures matérielles disponibles (CPU, CUDA ou MPS Apple Silicon). Mettez en place l'entraînement du modèle et discutez des stratégies de passage à l'échelle (par exemple, le parallélisme de données ou l'utilisation de bibliothèques HPC sous-jacentes comme `PETSc`).

10 Module 8 : Visualisation Scientifique pour Publication

Un chercheur doit savoir communiquer visuellement ses résultats de manière rigoureuse.

1. **Visualisation Statique de Production** : À l'aide de `Matplotlib` et `Seaborn`, générez une figure double panneaux montrant d'une part la solution approchée par le PINN sous forme de carte de chaleur (*heatmap*), et d'autre part l'évolution temporelle de l'erreur absolue de prédiction.

2. **Exigences Académiques** : Forcez l'intégration native de $\text{\LaTeX}$ pour le rendu des labels mathématiques via l'instruction `"text.usetex": True`. Exportez le graphique final exclusivement au format vectoriel `.pdf`.
 3. **Visualisation Interactive** : Implémentez une surface 3D interactive représentant $\hat{u}(x, t)$ à l'aide de `Plotly`, exportée dans un fichier HTML autonome pour intégration dans un tableau de bord de suivi de recherche.
-

11 Module 9 : Orchestration du Workflow et Validation Expérimentale

11.1 Exercice 9.1 : Automatisation via Snakemake

Afin de garantir que l'intégralité du pipeline (de la dérivation symbolique à la génération des figures d'édition) soit exécutable via une commande unique, écrivez le fichier d'orchestration **Snakefile**.

Ce fichier doit définir le graphe de dépendances acyclique dirigé (DAG) reliant les règles suivantes :

- **rule derive_symbolic** : Génère les fonctions résiduelles lambdifiées.
- **rule ingest_and_vectorize** : Filtre les données Parquet et prépare les matrices de discrétisation.
- **rule analyze_stability** : Évalue le conditionnement et la tolérance IEEE 754.
- **rule train_pinn** : Entraîne le réseau de neurones PyTorch/JAX et sauvegarde les poids du modèle.
- **rule generate_plots** : Produit les rapports visuels statiques (PDF) et interactifs (HTML).

11.2 Exercice 9.2 : Validation Expérimentale (Le Défi CI/CD)

Pour valider ce TP, vous devez provoquer volontairement une défaillance au sein de votre pipeline :

1. Introduisez une erreur de typage flagrante dans l'un de vos modules (par exemple, forcer une fonction qui attend un `np.ndarray` à recevoir une chaîne de caractères `str`).
2. Poussez le code sur GitHub et observez le blocage immédiat du pipeline lors de l'étape `mypy`.
3. Corrigez l'erreur, ajoutez un test unitaire limitant ce cas aux limites, et documentez la résolution au sein de vos messages de commit Git en respectant la norme des commits conventionnels (*Conventional Commits*).

11.3 Exercice 9.3 : Rapport Final et Interprétation Épistémique

Complétez le fichier `README.md` principal de votre projet. Ce document doit faire office de notice de réplication complète. Vous y incluez :

1. La procédure exacte de déploiement de l'environnement avec `uv`.
 2. L'explication physique et mathématique des résultats obtenus.
 3. Une analyse critique de l'impact du choix de la précision des flottants et des accélérations logicielles mises en oeuvre sur la fidélité de votre modèle d'IA Scientifique.
 4. Les badges de statut CI/CD et les liens vers les artefacts générés.
-

Récapitulatif des Livrables

- **Dépôt GitHub** : URL du dépôt (privé avec accès accordé ou public)
- **Structure complète** : Tous les fichiers sources, tests, configurations CI/CD, Snakefile
- **Documentation** : README.md complet avec procédure de réplication
- **Couverture de tests** : $\geq 90\%$ validée par pytest-cov
- **Qualité du code** : 0 erreur Ruff, 0 erreur Mypy strict
- **Pipeline CI/CD** : Fonctionnel avec badges de statut verts
- **Artéfacts scientifiques** : Figures PDF, modèles sauvegardés, rapports HTML